

Hospital Management System

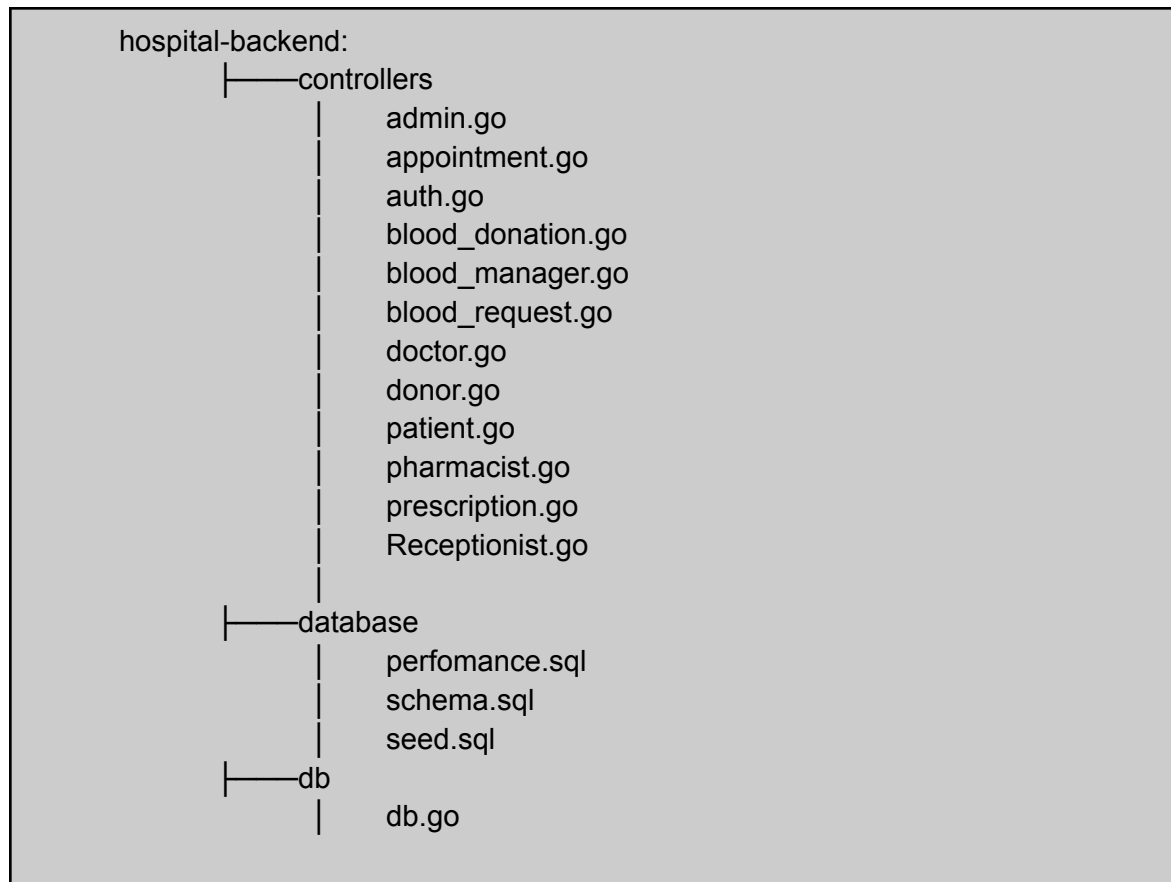
Backend API

We developed a REST API backend for a Hospital Management System using Go(Gin), PostgreSQL, and JWT authentication. Features include role-based access control across five user roles, transaction management with BEGIN/COMMIT/ROLLBACK, and optimized database interactions using raw SQL with connection pooling.

Tech Stack

- *Language*: Go 1.21+
- *Framework*: Gin
- *Database*: PostgreSQL 15+
- *Password Hashing*: bcrypt
- *Authentication*: JWT (golang-jwt/jwt)
- *Database Driver*: pgx/v5(pgx pool)
- *API Style*: REST(versioned at /api/v1/)

Project Structure



```
├── docs
│   ├── ACID.pdf
│   ├── ER_Diagram.pdf
│   └── schema.pdf
├── middleware
│   └── auth.go
├── models
│   └── models.go
├── routes
│   └── routes.go
├── .env
├── .env.example
├── go.mod
├── go.sum
├── main.go
├── Readme.md
└── swagger.yaml
```

Prerequisites

- Go 1.21+
- PostgreSQL 15+
- Git

Authentication

All endpoints ,except Register and Login, require a JWT token in the Authorization header:
Authorization: Bearer

Get token:

```
POST /api/v1/auth/login
Content-Type: application/json

{
  "email": "admin@hospital.com",
  "password": "admin123",
  "role": "admin"
}
```

Response:

```
{
  "message": "Login successful",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "role": "admin",
  "name": "Admin User"
}
```

Roles & Access

- admin : Full access to everything
- doctor : Prescriptions, blood requests, own schedule, patient history
- receptionist : Patient CRUD, appointment booking, ER management
- pharmacist : Dispense medicines, medicine stock, pending prescriptions
- blood_manager : Donor CRUD, blood donations, blood inventory, fulfill requests

API Endpoints Summary

Base URL: `http://localhost:8080/api/v1`

Auth

```
POST    /auth/register
POST    /auth/login
```

Patients : receptionist, admin

```
GET      /patients/
POST     /patients/
GET      /patients/:email
PUT      /patients/:email
DELETE   /patients/:email
```

Donors : blood_manager, admin

```
GET      /donors/
POST     /donors/
GET      /donors/:email
PUT      /donors/:email
DELETE   /donors/:email
```

Transactions

```
POST     /prescriptions/prescribe
POST     /dispensing/dispense
POST     /appointments/book
POST     /appointments/book-ot
POST     /blood/donate
POST     /blood-request/create
POST     /blood-fulfill/fulfill
```

Receptionist Dashboard : receptionist, admin

```
GET      /receptionist/appointments
GET      /receptionist/er-patients
POST     /receptionist/er-patients
GET      /receptionist/er-entries
POST     /receptionist/er-entries
POST     /appointments/available-doctors
```

Doctor Dashboard : doctor, admin

```
GET      /doctor/my-appointments
GET      /doctor/patient-history/:email
```

Pharmacist Dashboard : pharmacist, admin

GET	/pharmacist/pending-prescriptions
GET	/pharmacist/medicine-stock
GET	/pharmacist/medicine/:name
POST	/pharmacist/medicine

Blood Manager Dashboard : blood_manager, admin

GET	/blood-manager/donations
GET	/blood-manager/inventory
GET	/blood-manager/pending-requests
GET	/blood-manager/expired-blood

Admin

GET/POST	/admin/doctors
PUT/DELETE	/admin/doctors/:email
GET/POST	/admin/receptionists
PUT/DELETE	/admin/receptionists/:email
GET/POST	/admin/pharmacists
PUT/DELETE	/admin/pharmacists/:email
GET/POST	/admin/blood-managers
PUT/DELETE	/admin/blood-managers/:email

Transaction Scenarios

Transaction: Prescribe Medicines

doctor/admin

```
POST /api/v1/prescriptions/prescribe
```

Rollback triggers:

- Duplicate prescription same day : `trg_prescription_before`
- Duplicate medicine in same prescription : `trg_pm_before`
- Expired or non-existent medicine

Transaction: Dispense Medicines

pharmacist/admin

```
POST /api/v1/dispensing/dispense
```

Rollback triggers:

- Expired medicine : `trg_dispensing_before`
- Insufficient stock : `trg_dispensing_before`
- Auto-deducts stock on commit : `trg_dispensing_after_insert`

Transaction: Book Appointment

receptionist/admin

```
POST /api/v1/appointments/book
```

Rollback triggers:

- Doctor already has pending appointment at same date and time:
`trg_appointment_management`

Transaction: Book OT Appointment

receptionist/admin

```
POST /api/v1/appointments/book-ot
```

Rollback triggers:

- Doctor conflict
- No available OT room at requested slot
- Uses `SELECT FOR UPDATE` to prevent race conditions on OT assignment

Transaction: Record Blood Donation

blood_manager/admin

```
POST /api/v1/blood/donate
```

Rollback triggers:

- Donor donated within 90 days : `trg_donor_management_before`
- Uses `SELECT FOR UPDATE` on donor row (double-checked after lock)
- Blood expiry auto-set to `collected_date + 42 days`

Transaction: Create Blood Request

doctor/admin

```
POST /api/v1/blood-request/create
```

Rollback triggers:

- Patient already has a pending blood request

Transaction: Fulfill Blood Request

blood_manager/admin

```
POST /api/v1/blood-fulfill/fulfill
```

Rollback triggers:

- Request already fulfilled
- No compatible blood available for patient's blood group
- Insufficient units : `trg_bloo_fulfillment_before`
- Expired blood : `trg_blood_fulfillment_before`
- Incompatible blood type : `trg_blood_fulfillment_before`
- Uses `SELECT FOR UPDATE` on both `blood_request` and `blood` rows

Database

- **21 tables** : Admin, Doctor, Receptionist, Pharmacist, Blood_Manager, Patient, ER_Patient, Donor, Blood, Donation, OT, Appointment, Medicine, Prescription, Prescription_Medicine, Dispensing, ER_Shift, ER_Shift_Doctor, ER_Patient_Entry, Blood_Request, Blood_Request_Fulfillment
- **5 triggers** : trg_donor_management, trg_blood_inventory_management, trg_medicine_dispensing, trg_appointment_management, trg_er_shift_management
- **4 views** : Active_Blood_Inventory, Donor_Donation_History, ER_Shift_Staffing, Medicine_Expiry_Alert
- **4 indexes** : idx_er_doctor_availability, idx_blood_type_status, idx_blood_request_status, idx_medicine_name
- All IDS are serial

Changes in DataBase:

- IDs were previously NOT NULL Now changed to SERIAL type.
- An Admin Table was added.

Security Notes

- Passwords are hashed with bcrypt
- JWT tokens are signed with HS256 and are validated on every protected request
- RBAC middleware is used in this as per the instructions
- All the credentials were loaded from the env file.

Deployment On Cloud

- Opened the Port 8080 ON EC2 instance
- SSH it on windows using public IP.
- Installed GO and all its dependencies
- Installed Postgres and created a database;
- Pulled the repository on Cloud
clone <https://github.com/rabia-syed-103/Health-Care-Management-System.git>
- Navigated to Health-Care-Management-System
- Updated the .env file with new created database
- Run the server
go run [main.go](#)
- Tested using postman through the following base url
`http://<ec2-public-ip>:8080/api/v1/<endpoint>`